Les Notions de Base du Langage C

Cours complet et détaillé

Contents

1. Introduction et structure d'un programme C **2**
- 1.1 Cycle de vie d'un programme C . . . 2
- 1.2 Structure minimale . . . 3

2. Variables, types de données et constantes **4**
- 2.1 Déclaration et types de base . . . 4
- 2.2 Modificateurs de type . . . 4
- 2.3 Constantes . . . 4
- 2.4 Conversions de types (casting) . . . 4

3. Les opérateurs **5**
- 3.1 Arithmétiques . . . 5
- 3.2 Relationnels (renvoient 0 ou 1) . . . 5
- 3.3 Logiques . . . 5
- 3.4 Affectation et affectations composées . . . 5
- 3.5 Incrémentation / décrémentation . . . 5
- 3.6 Opérateurs bit à bit (utiles en sécurité/bas niveau) . . . 5
- 3.7 Opérateur ternaire . . . 6

4. Entrées / Sorties standard **7**
- 4.1 `printf` — affichage . . . 7
- 4.2 `scanf` — saisie . . . 7

5. Les structures de contrôle conditionnelles **8**
- 5.1 `if / else if / else` . . . 8
- 5.2 `switch` . . . 8

6. Les boucles **9**
- 6.1 `for` — nombre d'itérations connu . . . 9
- 6.2 `while` — condition testée avant chaque itération . . . 9
- 6.3 `do...while` — exécuté au moins une fois . . . 9
- 6.4 Contrôle de flux dans les boucles . . . 9
- 6.5 Boucles imbriquées . . . 9

7. Les tableaux **10**
- 7.1 Déclaration et initialisation . . . 10
- 7.2 Accès et parcours . . . 10
- 7.3 Tableaux à deux dimensions . . . 10
- 7.4 Tableau et pointeur . . . 10

8. Les chaînes de caractères **11**

8.1 Déclaration 11
8.2 Fonctions de <string.h> 11
8.3 Lecture d'une chaîne au clavier 11

9. Les fonctions **12**
9.1 Déclaration, définition, appel 12
9.2 Passage de paramètres 12
9.3 Fonctions sans retour (void) et récursivité 12

10. Les pointeurs **13**
10.1 Principe 13
10.2 Opérations courantes 13
10.3 Pointeurs et tableaux 13
10.4 Pointeur de pointeur 13

11. Les structures (struct) **14**
11.1 Définition et usage 14
11.2 Alias avec typedef 14
11.3 Structures et pointeurs 14
11.4 Structures imbriquées 14

12. Allocation dynamique de mémoire **15**
12.1 malloc, calloc, realloc, free 15
12.2 Trois erreurs classiques 15

13. Gestion des fichiers **16**
13.1 Ouverture, écriture, lecture, fermeture 16
13.2 Modes principaux 16

14. Le préprocesseur **17**

15. Portée des variables et durée de vie **18**

16. Bonnes pratiques et pièges classiques **19**

1. Introduction et structure d'un programme C

Le C est un langage **compilé**, **impératif**, **statiquement typé**, créé par Dennis Ritchie en 1972. Il est proche de la machine (accès mémoire direct via les pointeurs) tout en restant portable.

1.1 Cycle de vie d'un programme C

`Code source (.c) --> Préprocesseur --> Compilateur --> Assembleur --> Éditeur de liens --> Exécutable`

Commande typique avec `gcc` :

```
gcc -Wall -Wextra -o programme programme.c
./programme
```

`-Wall -Wextra` active tous les avertissements utiles (à toujours utiliser en apprentissage).

1.2 Structure minimale

```c
#include <stdio.h>   // directive d'inclusion : donne accès à printf, scanf...

int main(void)
{
    printf("Bonjour le monde\n");
    return 0;        // 0 = succès, valeur renvoyée au système d'exploitation
}
```

Points essentiels :

- `main` est le point d'entrée obligatoire.
- `main` peut être écrit `int main(void)` (aucun argument) ou `int main(int argc, char *argv[])` (arguments de la ligne de commande).
- Chaque instruction se termine par un `;`.
- Les blocs sont délimités par `{ }`.

2. Variables, types de données et constantes

2.1 Déclaration et types de base

Type	Taille (usuelle, 64 bits)	Exemple de valeur
`char`	1 octet	`'A'`
`int`	4 octets	`42`
`float`	4 octets	`3.14f`
`double`	8 octets	`3.14159265`
`short`	2 octets	`100`
`long`	8 octets (souvent)	`100000L`
`_Bool` (`stdbool.h`)	1 octet	`true` / `false`

```c
int age = 21;
float taille = 1.78f;
double pi = 3.14159265358979;
char lettre = 'P';
```

Possibilités de déclaration :

```c
int a;          // déclaration seule, valeur indéterminée
int b = 5;      // déclaration + initialisation
int c, d, e;    // déclarations multiples
int f = 1, g = 2; // déclarations + initialisations multiples
```

2.2 Modificateurs de type

- `signed` / `unsigned` : présence ou non du bit de signe.

```c
unsigned int compteur = 4000000000U; // uniquement positif, plage doublée
signed int temperature = -15;        // peut être négatif (comportement par défaut)
```

- `short` / `long` : ajustent la taille mémoire.

2.3 Constantes

Trois façons de définir une constante, chacune avec un usage différent :

```c
#define PI 3.14159        // constante du préprocesseur (remplacement textuel, pas de type)
const double E = 2.71828; // constante typée, vérifiée par le compilateur (recommandée)
enum { MAX = 100 };       // constante entière nommée, utile pour un ensemble de valeurs
```

2.4 Conversions de types (casting)

```c
int i = 10;
double d = (double) i / 3;   // conversion explicite : évite la division entière tronquée
int x = (int) 9.99;          // donne 9 (troncature, pas d'arrondi)
```

- **Conversion implicite** : le compilateur convertit automatiquement (ex : `int` vers `double`).
- **Conversion explicite (cast)** : imposée par le développeur, plus sûre et plus lisible.

3. Les opérateurs

3.1 Arithmétiques

```
int a = 7, b = 2;
a + b;  // 9
a - b;  // 5
a * b;  // 14
a / b;  // 3   -> division ENTIÈRE si les deux opérandes sont des int
a % b;  // 1   -> modulo (reste), uniquement sur des entiers
```

Cas particulier important : `7 / 2` donne 3 (troncature) alors que `7.0 / 2` donne `3.5`.

3.2 Relationnels (renvoient 0 ou 1)

```
a == b   // égalité
a != b   // différence
a > b, a < b, a >= b, a <= b
```

3.3 Logiques

```
(a > 0) && (b > 0)   // ET logique
(a > 0) || (b > 0)   // OU logique
!(a > 0)             // NON logique
```

Particularité : **évaluation paresseuse (short-circuit)** — dans `f() && g()`, si `f()` renvoie faux, `g()` n'est jamais exécuté.

3.4 Affectation et affectations composées

```
a = 5;
a += 3;  // équivalent à a = a + 3
a -= 2; a *= 2; a /= 2; a %= 2;
```

3.5 Incrémentation / décrémentation

```
int i = 5;
i++;   // post-incrémentation : utilise i puis l'incrémente
++i;   // pré-incrémentation : incrémente puis utilise i
```

Différence visible :

```
int x = 5;
int y = x++;  // y = 5, x devient 6
int z = ++x;  // x devient 7, z = 7
```

3.6 Opérateurs bit à bit (utiles en sécurité/bas niveau)

```
a & b   // ET binaire
a | b   // OU binaire
a ^ b   // OU exclusif (XOR)
```

```
~a      // complément à un
a << 2  // décalage à gauche (multiplie par 4)
a >> 1  // décalage à droite (divise par 2)
```

3.7 Opérateur ternaire

```
int max = (a > b) ? a : b;  // équivalent compact d'un if/else
```

4. Entrées / Sorties standard

4.1 `printf` — affichage

```c
printf("Age: %d, Taille: %.2f, Initiale: %c\n", age, taille, lettre);
```

Spécificateurs de format les plus courants :

Format	Type
`%d`	int
`%u`	unsigned int
`%f`	float/double
`%.2f`	double, 2 décimales
`%c`	char
`%s`	chaîne (char*)
`%x`	hexadécimal
`%p`	adresse (pointeur)
`%ld`	long

4.2 `scanf` — saisie

```c
int nombre;
printf("Entrez un nombre : ");
scanf("%d", &nombre);   // & obligatoire : scanf a besoin de l'ADRESSE de la variable
```

Piège classique : oublier le `&` provoque un comportement indéfini ou un crash. Cas particulier des chaînes : `scanf("%s", nom);` — pas de `&` car un tableau de `char` est déjà une adresse.

5. Les structures de contrôle conditionnelles

5.1 `if / else if / else`

```c
int note = 14;

if (note >= 16) {
    printf("Très bien\n");
} else if (note >= 14) {
    printf("Bien\n");
} else if (note >= 10) {
    printf("Passable\n");
} else {
    printf("Insuffisant\n");
}
```

5.2 `switch`

Utile quand une seule variable est comparée à plusieurs valeurs discrètes :

```c
int jour = 3;
switch (jour) {
    case 1:
        printf("Lundi\n");
        break;       // sans break, l'exécution continue vers le cas suivant (fall-through)
    case 2:
        printf("Mardi\n");
        break;
    case 3:
        printf("Mercredi\n");
        break;
    default:
        printf("Jour inconnu\n");
}
```

Possibilité utile : regrouper plusieurs cas sans instruction entre eux pour partager le même traitement :

```c
switch (jour) {
    case 6:
    case 7:
        printf("Week-end\n");
        break;
    default:
        printf("Jour ouvré\n");
}
```

6. Les boucles

6.1 `for` — nombre d'itérations connu

```c
for (int i = 0; i < 5; i++) {
    printf("%d ", i);
}
```

6.2 `while` — condition testée avant chaque itération

```c
int i = 0;
while (i < 5) {
    printf("%d ", i);
    i++;
}
```

6.3 `do...while` — exécuté au moins une fois

```c
int choix;
do {
    printf("Menu: 1-Jouer 2-Quitter\n");
    scanf("%d", &choix);
} while (choix != 1 && choix != 2);
```

6.4 Contrôle de flux dans les boucles

```c
for (int i = 0; i < 10; i++) {
    if (i == 3) continue;  // saute cette itération, passe à la suivante
    if (i == 7) break;     // arrête complètement la boucle
    printf("%d ", i);
}
```

6.5 Boucles imbriquées

```c
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        printf("(%d,%d) ", i, j);
    }
}
```

7. Les tableaux

7.1 Déclaration et initialisation

```c
int notes[5];                       // tableau non initialisé (valeurs indéterminées)
int notes2[5] = {12, 15, 8, 20, 10}; // initialisation complète
int notes3[5] = {12, 15};            // le reste est initialisé à 0
int notes4[] = {1, 2, 3};            // taille déduite automatiquement (3)
```

7.2 Accès et parcours

```c
notes[0] = 18;             // indices de 0 à taille-1
printf("%d", notes[0]);

for (int i = 0; i < 5; i++) {
    printf("%d ", notes2[i]);
}
```

Attention : le C ne vérifie pas les débordements d'indice (`notes[10]` compile mais provoque un comportement indéfini).

7.3 Tableaux à deux dimensions

```c
int matrice[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
printf("%d", matrice[1][2]); // affiche 6
```

7.4 Tableau et pointeur

Le nom d'un tableau se comporte comme un pointeur vers son premier élément :

```c
int t[3] = {10, 20, 30};
printf("%d", *(t + 1)); // équivaut à t[1] -> affiche 20
```

8. Les chaînes de caractères

En C, une chaîne est un **tableau de `char` terminé par le caractère nul `'\0'`**.

8.1 Déclaration

```c
char mot1[10] = "Bonjour";        // le compilateur ajoute automatiquement '\0'
char mot2[] = "Salut";            // taille déduite (6, avec le '\0')
char mot3[10] = {'H','i','\0'};   // construction manuelle
```

8.2 Fonctions de `<string.h>`

```c
#include <string.h>

strlen(mot1);              // longueur SANS compter le '\0'
strcpy(mot1, "Nouveau");    // copie une chaîne dans une autre (attention aux débordements)
strcat(mot1, mot2);         // concatène mot2 à la fin de mot1
strcmp(mot1, mot2);         // compare : 0 si égales, <0 ou >0 sinon
strncpy(mot1, mot2, 5);     // copie sécurisée limitée à n caractères
```

8.3 Lecture d'une chaîne au clavier

```c
char nom[50];
printf("Entrez votre nom : ");
scanf("%s", nom);          // s'arrête au premier espace
fgets(nom, 50, stdin);     // lit une ligne complète, y compris les espaces
```

9. Les fonctions

9.1 Déclaration, définition, appel

```c
int addition(int a, int b);   // prototype (déclaration), souvent dans un .h

int addition(int a, int b) {  // définition
    return a + b;
}

int main(void) {
    int resultat = addition(3, 4); // appel
    printf("%d\n", resultat);
    return 0;
}
```

9.2 Passage de paramètres

- **Par valeur** (par défaut) : la fonction reçoit une copie, ne modifie pas l'original.

```c
void incrementer(int x) {
    x = x + 1;   // n'affecte pas la variable de l'appelant
}
```

- **Par adresse (pointeur)** : permet de modifier la variable d'origine.

```c
void incrementer_ptr(int *x) {
    *x = *x + 1;
}
// appel : incrementer_ptr(&valeur);
```

9.3 Fonctions sans retour (void) et récursivité

```c
void afficherMessage(void) {
    printf("Message\n");
}

int factorielle(int n) {
    if (n <= 1) return 1;         // cas de base
    return n * factorielle(n-1);  // appel récursif
}
```

10. Les pointeurs

10.1 Principe

Un pointeur stocke une **adresse mémoire**.

```c
int a = 10;
int *p = &a;    // p contient l'adresse de a
printf("%d", *p);  // *p = "déréférencement" -> valeur pointée = 10
printf("%p", (void*)p); // affiche l'adresse elle-même
```

10.2 Opérations courantes

```c
*p = 20;    // modifie a via le pointeur -> a vaut maintenant 20
p = NULL;   // pointeur ne pointant vers rien (bonne pratique après libération)
```

10.3 Pointeurs et tableaux

```c
int t[3] = {1, 2, 3};
int *p = t;        // équivaut à &t[0]
p++;               // avance d'un élément (arithmétique de pointeur, pas d'un octet !)
printf("%d", *p);  // affiche 2
```

10.4 Pointeur de pointeur

```c
int a = 5;
int *p = &a;
int **pp = &p;
printf("%d", **pp);  // double déréférencement -> 5
```

11. Les structures (`struct`)

11.1 Définition et usage

```c
struct Etudiant {
    char nom[30];
    int age;
    float moyenne;
};

struct Etudiant e1 = {"Pape", 21, 15.5};
printf("%s a %d ans\n", e1.nom, e1.age);
```

11.2 Alias avec `typedef`

```c
typedef struct {
    char nom[30];
    int age;
} Personne;

Personne p1 = {"Fatou", 22};
```

11.3 Structures et pointeurs

```c
Personne *pp = &p1;
printf("%s", pp->nom);   // -> équivaut à (*pp).nom
```

11.4 Structures imbriquées

```c
typedef struct {
    int jour, mois, annee;
} Date;

typedef struct {
    char nom[30];
    Date naissance;
} Employe;

Employe emp = {"Moussa", {12, 5, 2000}};
printf("%d", emp.naissance.annee);
```

12. Allocation dynamique de mémoire

12.1 `malloc`, `calloc`, `realloc`, `free`

```c
#include <stdlib.h>

int *tab = malloc(5 * sizeof(int));   // alloue 5 int, valeurs indéterminées
if (tab == NULL) {                     // TOUJOURS vérifier l'échec d'allocation
    printf("Erreur d'allocation\n");
    return 1;
}

int *tab2 = calloc(5, sizeof(int));   // alloue ET initialise à 0

tab = realloc(tab, 10 * sizeof(int)); // redimensionne le bloc existant

free(tab);   // libère la mémoire
free(tab2);
tab = NULL;  // évite un pointeur "dangling" (pendouillant)
```

12.2 Trois erreurs classiques

- **Fuite mémoire** : oublier `free()`.
- **Double free** : appeler `free()` deux fois sur le même pointeur.
- **Utilisation après libération (use-after-free)** : déréférencer un pointeur déjà libéré.

13. Gestion des fichiers

13.1 Ouverture, écriture, lecture, fermeture

```c
#include <stdio.h>

FILE *f = fopen("donnees.txt", "w");  // modes : "r" lecture, "w" écriture (écrase), "a" ajout
if (f == NULL) {
    printf("Erreur d'ouverture\n");
    return 1;
}
fprintf(f, "Score: %d\n", 42);
fclose(f);

FILE *f2 = fopen("donnees.txt", "r");
char ligne[100];
while (fgets(ligne, 100, f2) != NULL) {
    printf("%s", ligne);
}
fclose(f2);
```

13.2 Modes principaux

Mode	Effet
`"r"`	lecture seule, le fichier doit exister
`"w"`	écriture, crée ou écrase le fichier
`"a"`	ajout à la fin, crée si absent
`"r+"`	lecture + écriture
`"rb"/"wb"`	mode binaire (images, structures brutes)

14. Le préprocesseur

```c
#include <stdio.h>       // inclusion de fichier
#define MAX 100          // macro constante
#define CARRE(x) ((x)*(x)) // macro fonction (toujours parenthéser !)

#ifdef DEBUG
    printf("Mode debug actif\n");
#endif

#ifndef MAX
    #define MAX 50
#endif
```

Piège classique des macros : `#define CARRE(x) x*x` donne un résultat faux pour `CARRE(2+3)` (devient `2+3*2+3 = 11` au lieu de 25). Toujours parenthéser.

15. Portée des variables et durée de vie

```c
int globale = 10;              // variable globale : visible dans tout le fichier

void fonction(void) {
    int locale = 5;            // variable locale : visible uniquement dans fonction()
    static int compteur = 0;   // conserve sa valeur entre les appels successifs
    compteur++;
    printf("%d\n", compteur);  // affiche 1, 2, 3... à chaque appel
}
```

- `extern` : permet d'utiliser une variable globale définie dans un autre fichier.
- `static` (au niveau fichier) : limite la visibilité d'une fonction/variable à ce fichier uniquement.

16. Bonnes pratiques et pièges classiques

1. Toujours vérifier la valeur de retour de `malloc`, `fopen`, `scanf`.
2. Initialiser ses variables avant utilisation.
3. Ne jamais confondre = (affectation) et == (comparaison) dans un `if`.
4. Toujours mettre un `break` dans un `switch` sauf fall-through voulu.
5. Éviter les débordements de tableaux (aucune protection automatique en C).
6. Libérer toute mémoire allouée dynamiquement (`free`).
7. Compiler avec `-Wall -Wextra` et lire les avertissements.
8. Utiliser `const` pour tout ce qui ne doit pas changer.

Fin du cours — passez maintenant au document d'exercices pour mettre en pratique chacune de ces notions.